Stat405

Dates and times

Hadley Wickham

cc BY NC

```
install.packages("lubridate")

# Note: always use the Austria mirror.  It's the
# main cran mirror so you'll always get the latest
# version of a package

library(ggplot2) # load first
library(lubridate) # load second!
```

1. Date-times
2. Time spans
3. Math with date-times

Date-times in R

Your turn

Riddle - When does:

one day != one day

one year != one year

Time is a deceptively complicated system to work with. To master it, we are going to learn about 4 new objects:

1. instants
2. intervals
3. periods
4. durations

Instant

An instant refers to a specific moment of time.

Instants are named by a unique combination of years, months, days, hours, minutes, seconds, and time zone elements.

e.g, 2011-3-28 08:44:32 CST

```
# lubridate turns strings into instants with
# functions that have y, m, and d in their
# names. Use the function whose name matches
# the order of the elements in the date

ymd("2011-03-28")
mdy("03/28/2011")
dmy("28032011")
ymd_hms("2011:03:28 08:50:30")

#order matters
ymd("2011-03-05")
ydm("2011-03-05")
```

Your turn

Parse the column of dates in `emails.csv` into date time objects.

Manipulating instants

```
# We can compare dates
now( ) > ymd("2011-03-29")

# round dates
floor_date(now( ), "month")

# and extract parts of dates
year(now( ))
```

Date component	Accessor
Year	`year()`
Month	`month()`
Week	`week()`
Day of year	`yday()`
Day of month	`mday()`
Day of week	`wday()`
Hour	`hour()`
Minute	`minute()`
Second	`second()`
Time Zone	`tz()`

```
now()
year(now())
hour(now())

day(now())
yday(now())
wday(now())
wday(now(), label = TRUE)
wday(now(), label = TRUE, abbr = FALSE)

month(now(), label = TRUE, abbr = FALSE)
```

Determine which day of the week you were born on

```
now()
year(now())
hour(now())

day(now())
yday(now())
wday(now())
wday(now(), label = TRUE)
wday(now(), label = TRUE, abbr = FALSE)

month(now(), label = TRUE, abbr = FALSE)
```

Your turn

Use the email data to determine at which hour of the day Professor Wickham is most likely to respond to your email.

```
# Accessor functions can also be used to change the
# elements of a date-time

time <- now()
year(time) <- 1999
hour(time) <- 23

day(time) <- 45

tz(time) <- "UTC"
```

```
# Accessor functions can also be used to change the
# elements of a date-time

time <- now()
year(time) <- 1999
hour(time) <- 23

day(time) <- 45

tz(time) <- "UTC"
```

Determine what day of the week your birthday will be on in 2012

Rounding instants

We can round an instant to a specified unit using floor_date(), ceiling_date(), round_date()

e.g.

```
round_date(now(), "day")
round_date(now(), "month")
```

Your turn

Use ddply to calculate how many emails Hadley received per day since September 1st. Plot the number of emails per day over time.

Compare the number of emails sent by day of the week.

```
# just the recent emails
recent <- subset(emails, time > ymd("2010-09-01"))

# binning into days
recent$day <- lubridate::floor_date(recent$time,
  "day")

# calculating daily totals
daily <- ddply(recent, "day", summarise,
  words = sum(words), emails = length(day))
qplot(day, emails, data = daily, geom = "line")
qplot(wday(day, label = T), emails, data = daily,
  geom = "boxplot")
```

Time spans

Consider

Suppose we want to use R to set our alarm

```
force_tz(ymd_hms("2011-01-01 08:30:00"),
"") + ddays(0:30)
```

how about

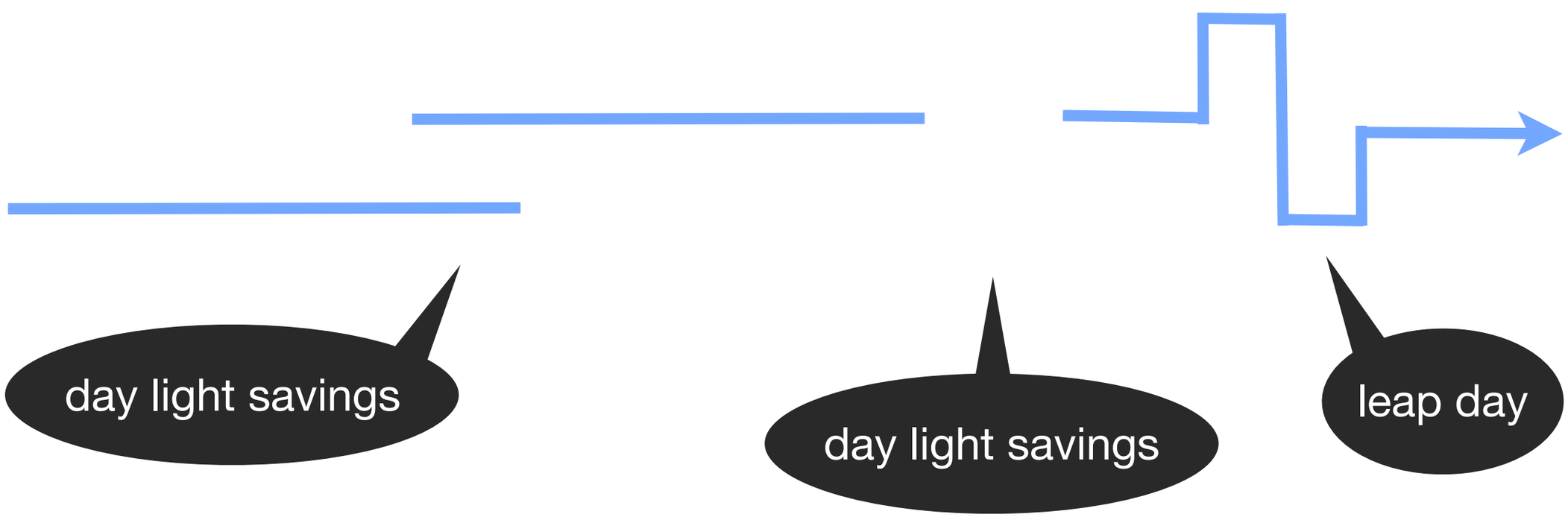
```
force_tz(ymd_hms("2011-03-01_08:30:00"),
"") + ddays(0:30)
```

What went wrong?

The number line

The number line

The time line (according to clock times)



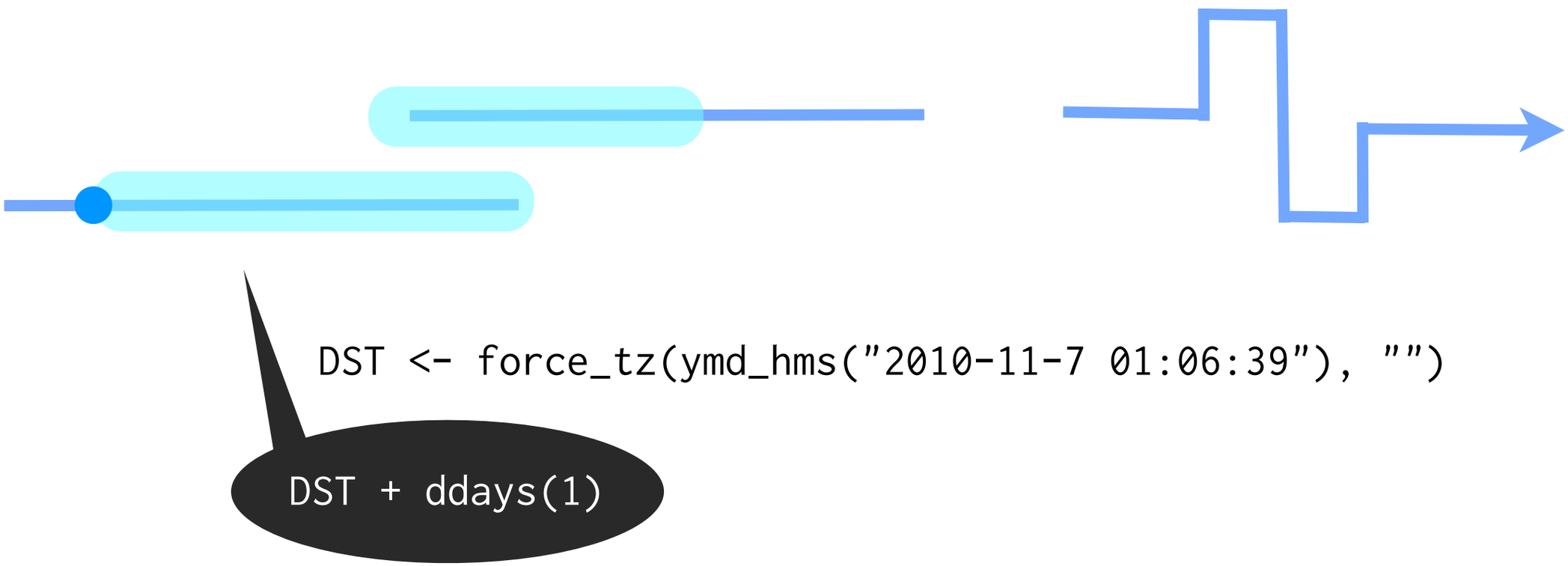
3 types of time spans

We can still do math on the time line, as long as we're specific about how we want to measure time.

1. durations (exact times)
2. periods (clock times)
3. intervals (specific times)

durations

Durations measure the exact amount of seconds that pass between two time points.

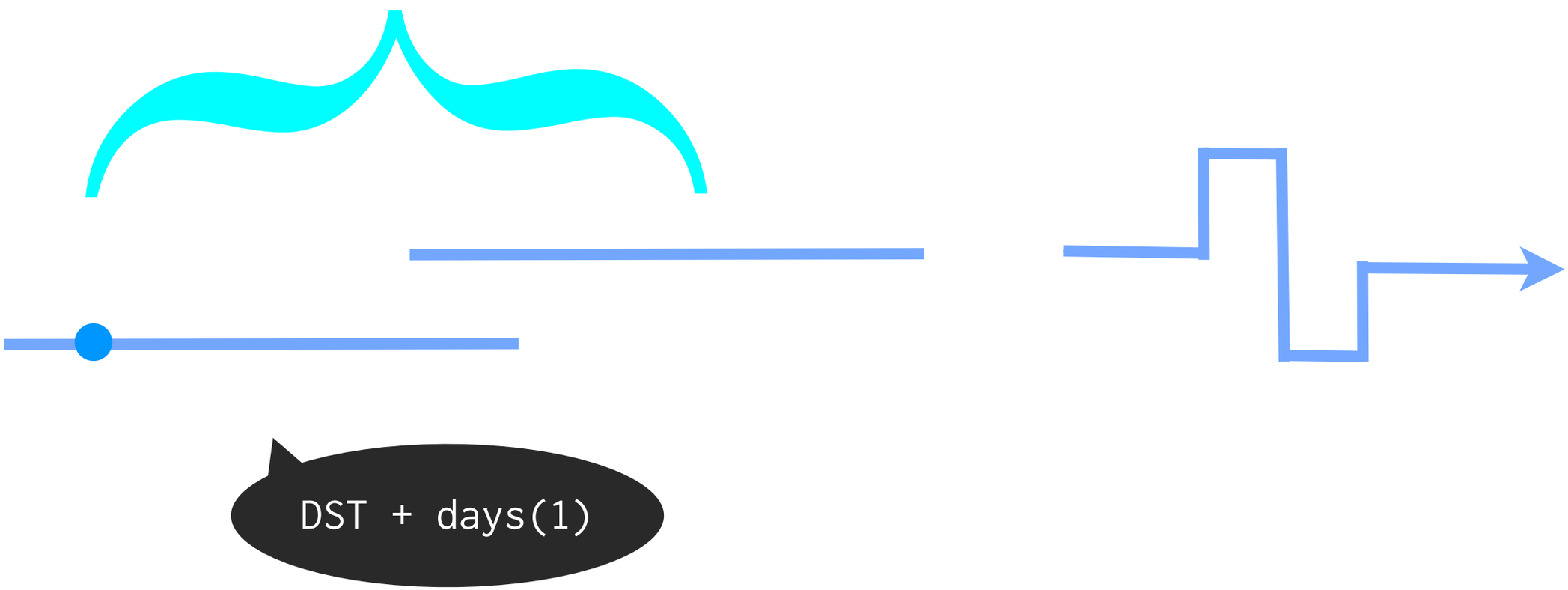


```
# Use d + the plural of of a unit of time to create
# a duration.

dminutes(5)
dhours(278)
dmonths(4) # why doesn't this work?
```

periods

Periods measure time spans in units larger than seconds. Periods pay no attention to how many sub-units occur during the unit of measurement.



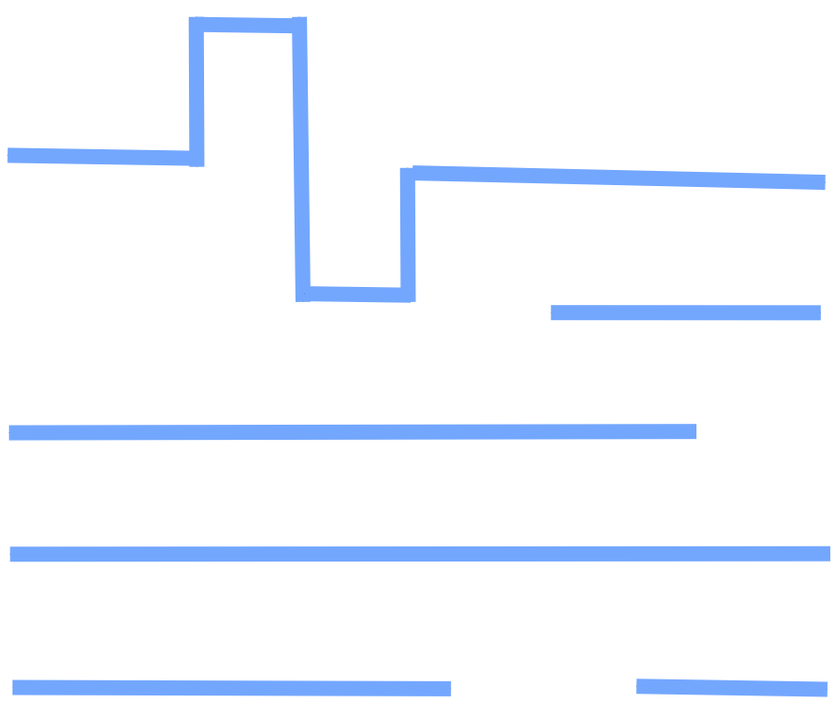
```
# Use the plural of a time unit to create a period.

minutes(5)
hours(278)
months(4) # months are not a problem
```

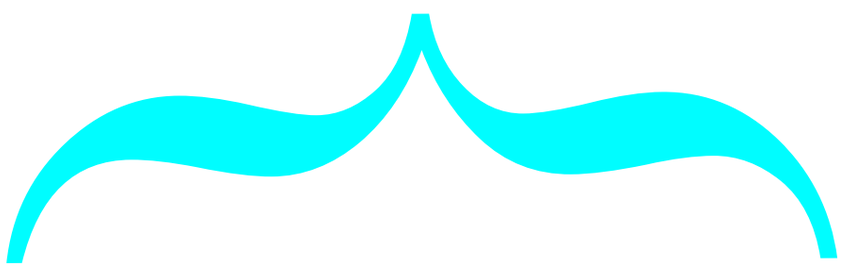
Why use periods?

No surprises.

`2010-11-01 00:00:00 + months(1)` will always equal 2010-12-01 00:00:00 no matter how many leap seconds, leap days or changes in DST occur in between

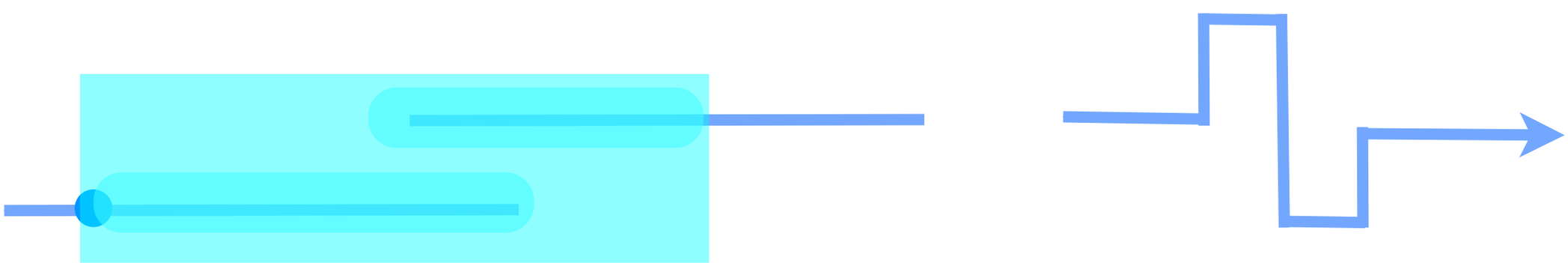


=



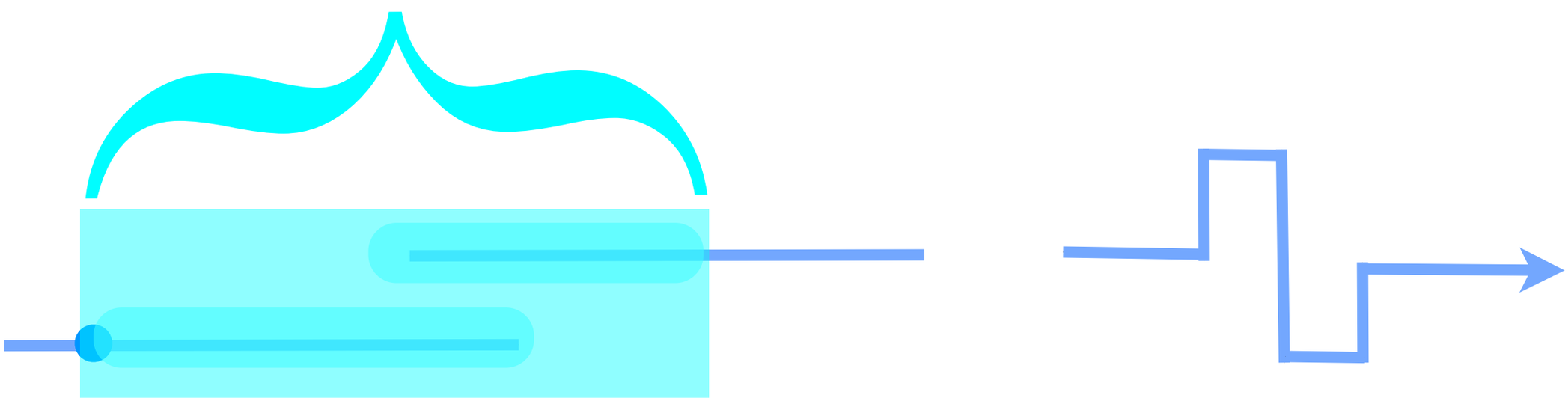
intervals

Intervals measure a time span by recording its endpoints. Since we know when the time span occurs, we can calculate the lengths of all the units involved.



intervals

Intervals measure a time span by recording its endpoints. Since we know when the time span occurs, we can calculate the lengths of all the units involved.



```
# To create an interval just subtract two dates.
int <- ymd("2010-01-01") - ymd("2009-01-01")

# Access and set the endpoints with start() and
# end().
start(int)
end(int) <- ymd("2010-03-14")

# Intervals are always positive

# Intervals can be accurately converted to either
# periods or durations with as.period() and
# as.duration()
```

Math with date-times

```
birthday <- ymd("1981-01-31")
life <- now() - birthday
life / ddays(1)

life / days(1)
life %/% days(1)
```

Your turn

Calculate your age in minutes. In hours. In days. How old is someone who was born on June 4th, 1958?

```
dob <- ymd("1979-10-14")
(dob - now()) %/% days(1)
(dob - now()) %/% months(1)
(dob - now()) %/% hours(1)
(dob - now()) %/% minutes(1)
```

Challenge

Write a function that takes a date and returns the last day of the month the date occurs in.

```
date <- today()
date - days(day(date) - 1) + months(1) - days(1)

# Note that date arithmetic is not commutative!
date - days(day(date) - 2) + months(1)

# OR

ceiling_date(date, "month") - days(1)
```